Week 12: Security of LLM Applications: Prompt Injection, Jailbreaks and Data Extraction

Machine Ethics and Artificial Intelligence Safety

Machine Ethics and Artificial Intelligence Safety (11118BLG001). Prof. Dr. Utku Kose. Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye. Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India). ORCID 0000-0002-9652-6415. E-mail: utkukose@sdu.edu.tr, utku.kose@und.edu, ukose@up.edu.mx, utkukose@gmail.com. This course is updated in line with current developments in the field. First version: September 2026. Last update: October 2026.

Overview

Language models are now embedded in applications that read e-mail, browse the web and call tools. This week examines the resulting attack surface, from direct and indirect prompt injection to jailbreaks and training data extraction. It also covers the layered defences and red teaming practices that reduce the risk [1, 2, 3].

Learning outcomes

By the end of the week, students are expected to explain why prompt injection follows from mixing instructions and data in one context. They should be able to distinguish direct from indirect injection and to describe the two failure modes of safety training behind jailbreaks. They should also explain training data extraction and membership inference. Finally, they should be able to design and evaluate layered defences for an LLM-integrated application.

Lecture

An attack surface made of text

An LLM-integrated application assembles a context from several sources: the developer's system prompt, the user's request and external content such as web pages, e-mails, documents and tool outputs. The model receives all of this as one sequence of tokens and has no reliable way to tell instructions from data. Perez and Ribeiro showed that simple handcrafted inputs could hijack the goal of a model or make it leak its hidden prompt, which they called goal hijacking and prompt leaking [4]. Greshake and colleagues went further with indirect prompt injection [1]. An attacker plants instructions in content that an application is likely to retrieve. The model then follows them without any action by the user. They demonstrated data theft and the manipulation of answers in real applications.

The OWASP Top 10 for LLM Applications, version 2025, lists prompt injection as the first risk [2]. It is followed by sensitive information disclosure, supply chain, data and model poisoning,

improper output handling and excessive agency. The list ends with system prompt leakage, vector and embedding weaknesses, misinformation and unbounded consumption. Figure 12.1 marks where several of these risks enter an application. Kose, Cankaya and Deperlioglu anticipated a future in which AI systems are both weapons and targets in cyber conflict [5], and LLM applications with access to tools make that anticipation concrete.

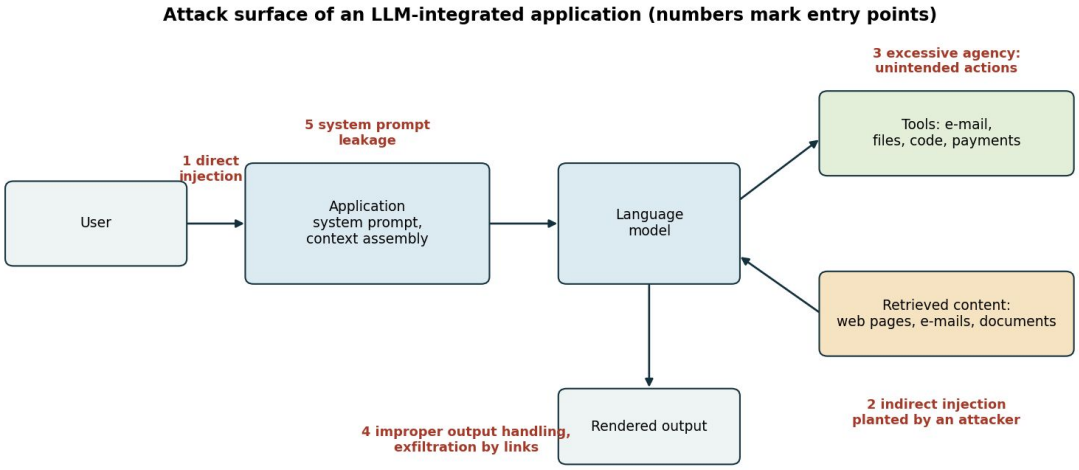


Figure 12.1. The attack surface of an LLM-integrated application. Numbered labels mark where direct and indirect injection, excessive agency, improper output handling and system prompt leakage enter [1, 2].

Jailbreaks and automated attacks

A jailbreak is an input that makes a model produce content its safety training was meant to prevent. Wei, Haghtalab and Steinhardt explained why safety training fails with two failure modes [6]. Competing objectives arise when a model's capabilities and its safety goals conflict, for example when a prompt demands a particular response format that leaves no room for refusal. Mismatched generalisation arises when safety training does not cover a domain in which the model is capable. An example is an instruction written in an encoding or a language that received little safety training. Attacks that combined both failure modes succeeded against the strongest models they tested.

Zou and colleagues automated the search: A gradient-guided optimisation found adversarial suffixes that caused open-weight models to comply with harmful requests, and the same suffixes transferred to closed commercial models [7]. The result links language model security to the adversarial examples of Week 8. Robustness cannot rest on the hope that attackers will not find the right input.

Privacy and data extraction

Models can reveal what they were trained on. Carlini and colleagues extracted hundreds of verbatim sequences from the training data of GPT-2 [8]. They generated many samples and ranked them with membership signals. The extracted text included personal names, phone numbers and e-mail addresses. They also found that larger models memorised more. Shokri and

colleagues introduced membership inference attacks, which decide whether a given record was part of a model's training set by training shadow models that imitate the target's behaviour [9]. Both attacks matter whenever a model is trained or fine-tuned on sensitive data, such as clinical notes or private correspondence.

Defence in depth and red teaming

No single defence stops prompt injection reliably, so practice relies on layers. Untrusted content can be marked and separated from instructions. Models can be trained to give priority to the instructions of the developer over those found in data. Tools can follow the principle of least privilege, so that an assistant that summarises e-mail cannot send it. Consequential actions can require human confirmation, and model output can be treated as untrusted input to downstream systems, for example by rendering links only from allowed domains [2]. Detectors for injected instructions help, but they generalise poorly to paraphrases and to other languages, as the Colab notebook shows.

Red teaming searches for failures before attackers do. Perez and colleagues used one language model to generate test cases for another and found many harmful behaviours at a scale that manual testing could not reach [3]. Ganguli and colleagues released a large data set of human red-team attacks [10]. Models trained with reinforcement learning from human feedback became harder to red-team as they grew. Other model types did not show this trend.

> ***Pause and reflect.*** *An assistant reads a university inbox and can send e-mail. Which single design change would remove the largest share of the injection risk, and what would it cost the users?*

Hands-on components

Interactive lab: A scripted toy assistant, not a real language model, summarises e-mails. Some e-mails contain injected instructions. Switch defensive layers on and off to see which attacks still succeed, which legitimate e-mails are wrongly blocked and what each layer costs [1, 2, 6]. Link: https://utkukose.github.io/machine-ethics-ai-safety-NB-lecture/weeks/week-12/lab.html

Colab notebook: It rebuilds the toy assistant in Python and evaluates all 64 combinations of six defensive layers. It then trains a small detector for injected instructions and tests it on paraphrased and Turkish attacks [1, 2]. Link: https://colab.research.google.com/github/utkukose/machine-ethics-ai-safety-NB-lecture/blob/main/weeks/week-12/NB12_llm_security.ipynb

Reflection prompts

1. Map an LLM application you use or build to Figure 12.1. Which entry point is the least protected?

2. Why did the detector fail on Turkish injections? What does that imply for systems deployed in multilingual settings?
3. What is the difference between red teaming and a penetration test of ordinary software? Name one similarity and one difference.

Weekly task and submission

What to do. Write a threat model and a defence plan for one language model application in your field. The application must read external content and be able to call at least one tool. First, describe the assets, the attackers and the entry points. Second, map each risk to an item of the OWASP Top 10 for LLM Applications 2025. Third, propose at least four defensive layers and state the cost of each. Fourth, describe how you would red-team the application.

Where to look. Lecture: sections "An attack surface made of text" and "Defence in depth and red teaming". Interactive lab: the toy e-mail assistant and its defensive layers (tab: Explore). Colab notebook: Section 1 for combinations of layers and Section 2 for the blind spots of a learned detector.

Example. An assistant that summarises supplier e-mails and can create calendar events is exposed to indirect prompt injection, the first item of the OWASP list. Its layers could include marking external text as data, an injection detector, a tool allowlist and human confirmation of new events.

What to submit. About 500 words with a table of the risks, and the Colab notebook with Exercises 1 and 2 completed.

The weekly task supports self-learning and builds a personal portfolio. In an active semester, students who follow the course with the instructor can send their work for evaluation to utkukose@sdu.edu.tr or utkukose@gmail.com. The e-mail contains the items that the task lists under What to submit. It also includes the learning log exported from the interactive lab (tab: Reflect and submit).

References

[1] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23) (pp. 79-90). ACM. https://arxiv.org/abs/2302.12173

[2] OWASP GenAI Security Project (2024). OWASP Top 10 for LLM Applications 2025 (version 2025, released 18 November 2024). OWASP Foundation. https://genai.owasp.org/

[3] Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N., & Irving, G. (2022). Red teaming language models with language models. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (pp. 3419-3448). Association for Computational Linguistics. https://arxiv.org/abs/2202.03286

[4] Perez, F., & Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. arXiv preprint arXiv:2211.09527. https://arxiv.org/abs/2211.09527

[5] Kose, U., Cankaya, S. F., & Deperlioglu, O. (2017). Cyber wars under the shadow of artificial intelligence: A future perspective. International Journal of Engineering Science and Application, 2(2), 71-76.

[6] Wei, A., Haghtalab, N., & Steinhardt, J. (2023). Jailbroken: How does LLM safety training fail?. In Advances in Neural Information Processing Systems 36. https://arxiv.org/abs/2307.02483

[7] Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., & Fredrikson, M. (2023). Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043. https://arxiv.org/abs/2307.15043

[8] Carlini, N., Tramèr, F., Wallace, E., et al. (2021). Extracting training data from large language models. In 30th USENIX Security Symposium (pp. 2633-2650).

[9] Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017). Membership inference attacks against machine learning models. In 2017 IEEE Symposium on Security and Privacy (SP) (pp. 3-18). IEEE. https://doi.org/10.1109/SP.2017.41

[10] Ganguli, D., Lovitt, L., Kernion, J., et al. (2022). Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. arXiv preprint arXiv:2209.07858. https://arxiv.org/abs/2209.07858